Enterprise RAG - Tech Stack and Build Status

Free-tier deployment on a Google Cloud VM (about 1 GB RAM). Hosted services do the heavy work, so the VM only runs the API, SQLite and local files.

Updated to match the code in the enterprise-rag folder. Where the built system differs from the original design, the text says so; items marked Not built are still design only.

0. Build status

Area	State	Verified how
Auth, roles, users, SQLite schema	Built	Automated tests (pytest, 75 pass in total)
Ingestion: parsers, versioning, chunking, PII, embed, BM25, Qdrant	Built	Tests with fake providers; Qdrant in-memory smoke test; real BM25 and Presidio smoke tests
Retrieval and chat: hybrid search, relevance gate, rerank, guardrails, citation check, SSE streaming, signed image URLs	Built	Tests with fake LLM and store; UI driven in a browser against a fake backend
Evaluation harness (recall, MRR, citations, refusals, latency, red-team, threshold gate, RAGAS export)	Built	Tests; RAGAS itself not run; no golden set written yet (example file only)
React UI: login, chat, documents and upload, users, feedback buttons	Built	Browser check and production build
Postman collection	Built	newman run: 32 assertions, 0 failed (against the fake backend)
Langfuse tracing and user feedback	Built	Tests with the real SDK and a mock server; not tried on Langfuse Cloud
Deploy scripts (installer, systemd, Caddy, backup and restore)	Written	Backup and restore round trip tested locally; installer, systemd and Caddy not run on a real VM
Real OpenAI, Groq and Qdrant Cloud calls	Not tried	No keys used so far. Model names in config.yaml are placeholders to confirm
1 GB VM memory	Not measured	Section 6 numbers are still estimates

1. Decisions taken

Topic	Decision
LLM and embeddings	Groq for the LLM (answers, rerank, judge); OpenAI for embeddings and vision (image captioning and OCR); each is a separate config setting behind its own interface
Interface	React web UI (chat + document upload) on a FastAPI REST backend (Swagger docs); one backend process to fit in 1 GB
Vector database	Qdrant Cloud free tier, hosted, not on the VM
Observability	Langfuse free tier (cloud)
Metadata, audit, eval results	SQLite on the VM disk
File storage	Local folder on the VM instead of S3 (data/raw, data/assets, data/captions)
Best practices in scope	Evaluation harness, governance basics, hybrid search with reranking
Input file types	PDF, DOCX, XLSX/CSV, HTML, Markdown, and standalone images (PNG/JPG), not only the sample PDF
Answer output	Text with citations plus the source images used, served from the local assets folder
API testing	Postman collection included, plus Swagger docs
Authentication	JWT login; roles Admin, Ingestor, User; users stored in SQLite; no self-signup
Out of scope for v1	Multi-tenant isolation (no tenant_id), agents

2. Stack by layer

Layer	Choice	Why
API	FastAPI + Uvicorn (1 worker)	Typed, async, auto Swagger; small footprint
Frontend	React 18 + TypeScript + Vite 5, React Router (hash routes), TanStack Query, react-markdown (no raw HTML)	Built to static files on your PC or CI; no Node process on the VM. Bundles live under /static because the API owns /assets
Chat transport	Server-Sent Events from FastAPI for streamed answers	Words appear as they are generated; simpler than WebSockets. Events: token, replace, final. Tokens go out before the citation check, so a replace event can correct the text
Config	pydantic-settings + .env	Keys out of code; one settings object
Auth	PyJWT (HS256 access tokens), argon2-cffi password hashing, role checks as FastAPI dependencies	Small and well maintained; no extra service; argon2 memory cost set low for a 1 GB VM
Parsing	PDF: PyMuPDF (text, page render, images) + pdfplumber (tables). DOCX: python-docx. XLSX: openpyxl. CSV: stdlib csv. HTML: BeautifulSoup. Markdown: header splitter. Images: Pillow then OpenAI vision.	Light, no system packages; one parser per file type feeding the same chunk format; OpenAI vision handles scans and charts
Vision / OCR	OpenAI vision-capable small model via the openai SDK (model name pinned in config)	Captions for charts, photos, scanned pages; no Tesseract install
Chunking	Structure-aware by block type; recursive splitter inside text blocks (size 1000, overlap 15 percent, both in config); tables kept whole with a summary	Per-content-type routing; only langchain-text-splitters is used from LangChain
Embeddings	OpenAI text-embedding-3-small with the dimensions parameter set to 768 (default is 1536)	No VM RAM; smaller vectors fit the Qdrant free tier
Sparse / keyword	BM25 sparse vectors via fastembed	Helps numbers, clause IDs and names that dense search misses
Vector DB	Qdrant Cloud: named dense + sparse vectors, payload indexes	Hybrid search with RRF fusion done server-side
Reranking	Groq small model listwise rerank, top 20 to top 6 (on/off switch)	A local cross-encoder will not fit in 1 GB
Generation	Groq (OpenAI-compatible API, groq SDK); context-only prompt with [S1] citations	Grounded answers with citations to document and page
Guardrails	Input, retrieval (relevance gate), context and output guards written in plain Python (section 5d)	Governance basics without extra services
PII	Microsoft Presidio with a small spaCy model, applied at three points (section 5d)	Open source, runs locally so no text goes to a third party; RAM to be measured on the VM
Metadata store	SQLite (aiosqlite)	Documents, versions, chunks, audit log, eval runs
File store	Local disk: data/raw, data/assets, data/captions	Replaces S3; paths stored in SQLite and the Qdrant payload
Tracing	Langfuse SDK 5.x: trace query (spans guard, embed_query, retrieve, rerank, generate, output_check) and trace ingest (parse, caption, chunk, pii, embed_and_upsert); scores on each answer; thumbs up/down	Latency, time to first token, chunk IDs, scores. No text sent unless tracing.log_content is on; then masked. Never breaks an answer
Reliability	tenacity retry with backoff on 429/503	Free tiers (Groq, and Gemini in earlier testing) return rate-limit and overload errors
Evaluation	pytest + custom harness in evaluation/ (recall@k, MRR, citation and refusal checks, red-team, threshold gate); RAGAS via JSONL export on the dev PC or CI (section 5g)	Page 13 of the sample PDF gives 5 ready golden questions; RAGAS is too heavy for the 1 GB VM

Providers (decided): embeddings = OpenAI, vision/OCR = OpenAI, LLM = Groq. They are three independent settings (embedding.provider, vision.provider, llm.provider), each behind its own interface, so any can be swapped later. Embeddings cannot be mixed in one Qdrant collection: changing the embedding model or dimension means

a new collection and a full re-embed. Groq has no embedding models. Check Groq's and OpenAI's current model lists and pricing before pinning names; OpenAI has no free tier. Gemini is no longer used.

3. Ingestion flow

1) Upload to FastAPI, save to data/raw, compute SHA-256. 2) Look the hash up in SQLite: same hash means skip; same document ID with a new hash means a new version that replaces the old chunks. 3) PyMuPDF and pdfplumber extract text and tables; image-only pages and figures are rendered and sent to OpenAI vision for OCR and captions. 4) Chunk by content type and attach metadata: document_id, version, page, type, section, asset path, file_sha256, chunk_sha256. 5) Embed dense (OpenAI) and sparse (BM25) and upsert into Qdrant. 6) Record document, chunks and status in SQLite. 7) Send one Langfuse trace for the whole run.

4. Query flow

1) Input checks. 2) Embed the question (dense + sparse). 3) Qdrant hybrid query with RRF fusion and payload filters (document, type, page range). 4) Optional Groq rerank keeps the best 6. 5) Pick the final chunks and the local image files for figure hits (Not built: neighbor-chunk expansion and whole-table expansion). 6) Build the prompt with source tags. 7) Groq answers with citations. 8) Output check: every citation must map to a retrieved chunk. 9) Return the answer, the citations, and a list of image URLs for the figures, charts or photos that were used; images are served by FastAPI from data/assets through signed links that expire after 5 minutes. 10) Write an audit row (query, chunk IDs, model, latency) to SQLite and a Langfuse trace with scores.

5. Data model

Where	What is stored
SQLite: users	user_id, username, password_hash, role (admin / ingestor / user), is_active, created_at
SQLite: documents	document_id, uploaded_by, filename, file_sha256, version, status, created_at, raw_path
SQLite: chunks	chunk_id, document_id, version, page, type, chunk_sha256, asset_path, access_group, pii_flags
SQLite: audit_log	timestamp, user, query, chunk_ids, model, tokens, latency_ms
SQLite: eval_runs	run_id, config, recall_at_k, answer_score, created_at
Qdrant payload	document_id, version, page, type, section, filename, file_sha256, chunk_sha256, asset_path, access_group (placeholder, unused in v1), pii_flags, text
Local disk	data/raw (originals), data/assets (cropped images, page renders), data/captions (cached vision output)

5b. Roles and permissions

Action	Admin	Ingestor	User
Log in, view own profile	Yes	Yes	Yes
Ask questions, see citations and source images	Yes	Yes	Yes
List documents and their status	Yes	Yes	Yes
Upload, re-ingest, replace a document	Yes	Yes	No
Delete a document and its chunks	Yes	No	No
Create, disable users, change roles	Yes	No	No
View audit log and Langfuse trace links	Yes	No	No
Run evaluation harness, see eval results (command line, results in eval_runs)	Yes	No	No
Rate an answer (own answers only)	Yes	Yes	Yes

Design notes: the first Admin is created from a command-line command, there is no public signup, and only an Admin creates users. Access tokens are short-lived (for example 30 minutes) and carry user_id and role. Because a JWT cannot be revoked, every request also checks the SQLite is_active flag, so a disabled user is blocked at once. Images are served through an authenticated endpoint; since a browser image tag cannot send an

Authorization header, the API returns short-lived signed image URLs. Login is rate-limited and the VM needs HTTPS (a reverse proxy such as Caddy) so tokens are not sent in clear text. The JWT secret lives in .env, never in the code.

5c. User interface (React)

Screen	Who sees it	What it does
Login	Everyone	Username and password; shows the role after login
Chat	Admin, Ingestor, User	Ask questions, streamed answer with Markdown, citation chips [S1], source images, document and type filters, Helpful and Not helpful buttons. Not built: source side panel, copy button
Documents	Admin, Ingestor, User (read); Admin delete	Table of documents with type, version, status, uploaded by; filter and search
Upload	Admin, Ingestor	Drag and drop (PDF, DOCX, XLSX, CSV, HTML, Markdown, images), file type and size check, per-file progress, status polling (queued, parsing, captioning, embedding, done or failed), duplicate and new-version notice from the SHA-256 check
Users	Admin	Create or disable users and change roles
Audit and evaluation	Admin	Not built in the UI: audit rows are in SQLite, evaluation runs from the command line, traces in Langfuse

Endpoints as built: GET /health, POST /auth/login, GET /auth/me, POST /users (admin), POST /documents (admin or ingestor; returns at once, ingestion runs as a background task), GET /documents, GET /documents/{id} (versions and status), DELETE /documents/{id} (admin), POST /chat, POST /chat/stream (SSE), POST /chat/feedback and GET /assets/... (signed image links). The UI hides what a role cannot do, but the backend enforces every permission. Design change: the access token is kept in sessionStorage (cleared when the tab closes), with no refresh cookie; this is simpler but a script injected into the page could read it, which is why the CSP is strict and Markdown is rendered without raw HTML. FastAPI serves the built UI from the same origin, so there is no CORS and no Node process on the VM. Not built: GET /sources, admin routes for audit and evaluation.

5d. Guardrails and PII (added from the flow guide)

Stage	Check	Action on failure
Input	Prompt-injection and jailbreak patterns, off-topic requests, PII, length limit	Reject, or mask and continue; log the event
Retrieval (relevance gate)	Top score below a threshold	Answer 'I could not find this in the documents' and skip the LLM call
Context	Retrieved text can hide instructions (indirect injection)	System prompt: treat document text as data, never as commands; strip suspicious patterns
Output	Grounded in the context, every citation maps to a retrieved chunk, no PII, no unsafe content	Regenerate once, then fall back to a refusal

PII with Presidio, mode off, flag or mask in config (default flag): (1) at ingestion, before chunks are embedded or stored, if the use case allows it; (2) before the question and context are sent to Groq; (3) on the answer before it is returned. Also masked before the audit log and Langfuse (mask callable in the SDK). In a smoke test Presidio missed one fake phone number (212-555-0198), so treat it as a safety net, not a guarantee. If PII is masked before storing, the original values stay only in the local raw file, so decide per use case. Presidio needs a spaCy model; use the small one and measure RAM before relying on it. Decision: Presidio plus plain Python checks; Guardrails AI is not used in v1.

5e. Chunking, memory and retrieval tuning (added from the flow guide)

Topic	Practice
Chunk size and overlap	Use 10 to 20 percent overlap (the guide notes 6 percent is low). Compare sizes and overlaps on the golden set with recall@k and MRR instead of guessing.

Topic	Practice
Parent-child and MMR	Parent-child for long documents (search small chunks, send the larger section). MMR to avoid several near-duplicate chunks.
Embedding dimension	Chunk size is how much text one vector covers; dimension is how many numbers describe it. Reduced dimension cuts storage, not API price. Keep model name and dimension stored with the collection.
Qdrant memory	Scalar int8 quantization (about 4x less RAM) with rescoring and oversampling; keep full vectors and large payload text on disk; index only the fields you filter on.
Collection naming	One collection per embedding model and dimension (built as erag__; the app refuses to run against a mismatched collection) so a model change is a blue/green migration, since embeddings are not portable between models.
Metadata schema	Decide the full payload now: adding fields later means re-ingesting everything.
Filtered search	Measure recall@k with the filter switched on, because strict filters can lower recall in approximate indexes.
Fewer vectors	Drop headers and footers, deduplicate by chunk hash, keep tables whole.

5f. Evaluation, observability and alerts (added from the flow guide)

Layer	What to measure	How
Retrieval	Recall@k, hit rate, MRR	Golden set of 30 to 50 questions, each with the correct file and page; compare chunk sizes, dimensions, quantization
Generation	Faithfulness, answer relevance, citation accuracy	LLM-as-judge with a Groq model, spot-checked by a human on a sample
Safety	PII leakage, injection success	Adversarial question set run on every release
Regression gate	No metric below the agreed threshold	Run the harness in CI on any change to prompts, chunking or models
Online	Quality on real traffic	Score a sample of traces (about 10 percent); thumbs up/down goes to Langfuse and bad runs grow the golden set

Langfuse as built: one trace per request (query) and per upload (ingest) with the spans listed in section 2. Scores on every answer: found, top_dense, citation_valid, regenerated, gated, blocked; the thumbs buttons add user_feedback through POST /chat/feedback, which only accepts a rating from the user who asked (checked through the trace ID stored in audit_log). User IDs are hashed. Without both Langfuse keys tracing does nothing. Not built: LLM-judged online scoring of a sample, nightly eval as Langfuse experiments, prompt management, alerts.

Alert (production; the POC only reviews dashboards)	Example threshold
Daily LLM spend	Above 150 percent of the 7-day average
Error rate	Above 2 percent over 15 minutes
p95 latency	Above 8 seconds over 15 minutes
Faithfulness score	Online average below 0.8
Zero-result rate	Above 10 percent of queries
Guardrail blocks	Sudden spike in injection or PII blocks

5g. Evaluation, cost and reliability (added from the enterprise RAG design)

Topic	Practice
Ports and providers	Built: LLM, embedder and vision sit behind interfaces chosen in config/config.yaml, with secrets in .env, validated at startup. There is no dev/vm profile switch: the same config is used with different .env values and tracing.environment.
Guardrail policy	Thresholds, topic lists and PII entity types live in one versioned policy file, reviewed like code. If a guard errors or times out, fail closed (refuse) and raise a metric.

Topic	Practice
Red-team set	30 to 50 adversarial prompts (injection, data extraction, off-topic, toxic) run in CI; track block rate and false-block rate.
PII at logging	Mask text before it goes to Langfuse or logs, hash user IDs, never log raw documents, and use deterministic placeholders such as PERSON_1 so masked text stays readable.
RAGAS metrics	Context precision, context recall, faithfulness (strict gate), answer relevancy, answer correctness (offline). Custom: citation accuracy, refusal correctness, guardrail rates.
Golden set and judge	40 to 60 questions covering text, table, image and chart questions plus unanswerable and adversarial ones; generate candidates, then correct by hand; keep in Langfuse Datasets and version control. Judge is a different model from the answering model, temperature 0, fixed between runs; pin the RAGAS version.
Eval cadence	Partly built (python -m evaluation.run --limit 15 is the gate; nightly Langfuse experiments and online scoring are not). Design: about 15 questions on every pull request as a gate (fail on a faithfulness or citation drop), full set nightly as a Langfuse experiment, about 5 percent of live traces scored online. Any change to chunking, embedding model, prompt, reranker or guardrail policy must show an eval comparison first.
Prompts	Not built. Design: versioned prompts with labels (dev, staging, prod) in Langfuse Prompt Management; each trace links the prompt version used.
Tracing resilience	Tracing is asynchronous and non-blocking; the app keeps working if Langfuse is down.
Cost controls	Content-hash chunk IDs so unchanged chunks are never re-embedded; cache query embeddings and repeated answers; cap context and answer length; per-user daily request and token limits; caption each image once and skip decorative ones; small model by default and escalate only when the grounding check fails; Langfuse cost per request, tracked against eval scores (cost per correct answer).
Reliability	Timeouts, bounded retries with backoff, model fallback, dead-letter handling for failed ingestion jobs, and an embedding guard: collection names include model and dimension and the app refuses to query a mismatched collection.
Conversation memory	Keep a short window of recent turns, summarise past a token limit, and store session IDs rather than raw personal text.
Access group field	Keep an access-group field in chunk metadata now, so document-level filtering can be added later without re-ingesting (decided: added as a placeholder, unused in v1, no tenant_id).

Notes: the enterprise design targets AWS with Bedrock; this POC runs Groq, OpenAI and Qdrant on a free-tier VM, so the same patterns apply but with different adapters. RAGAS has a large dependency tree and its judge calls spend Groq free-tier quota, so run it on the dev PC or in CI, not on the 1 GB VM. Risks it lists: Langfuse free-tier limits during evals (sample traces and check current limits), local versus managed guardrail parity (share one red-team set), same-family judge bias (use a different judge and spot-check by hand), and a small golden set giving noisy scores (grow it from real failures every week).

6. Memory budget (my estimates, measure on the VM)

Item	Approx. RAM
OS and SSH	150-250 MB
Python, FastAPI, Uvicorn	80-150 MB
PyMuPDF, pdfplumber, Pillow while parsing a PDF	100-250 MB peak
fastembed BM25 (onnxruntime import)	80-150 MB
Langfuse and client libraries	40-80 MB

This is tight against 1 GB. Add 1-2 GB of swap, process one document at a time, and render pages at moderate DPI.

6b. Deployment as built (deploy/)

Piece	Details
Installer	install.sh for Debian or Ubuntu: 2 GB swap, app user erag, Python venv, systemd unit, Caddy from the distro package. Re-running updates code and never touches .env or data
API service	One uvicorn worker on 127.0.0.1:8000, MemoryMax 800M, hardened systemd options
HTTPS	Caddy with automatic certificate, security headers and CSP, streaming not buffered, 25 MB upload cap; ports 80 and 443 only
Backup	Nightly timer keeps 7: consistent SQLite copy plus raw, assets and captions. Optional age encryption and Google Cloud Storage copy. Qdrant vectors are not backed up (re-ingest to rebuild)
Bundle	make-bundle.sh builds the UI and packs the code into a tgz to copy to the VM

Status: backup and restore tested locally; everything else in this table has not been run on a real VM.

7. Known limits and risks

Risk	Mitigation
Groq free-tier rate limits and model retirements; OpenAI has no free tier, so embeddings and vision cost money (small per call, but cap usage)	Retry with backoff, cache captions on disk, model names in config, per-user limits, a fallback model
Free-tier prompts may be used by the provider, and document text is sent to OpenAI for embeddings and vision	Use sample data only; do not upload real personal or client data
Qdrant free cluster is small	768-dim vectors, no duplicate chunks, delete old versions on re-upload
Local disk and SQLite are lost if the VM is deleted	Nightly copy of data/ and the .db file to another place you control
One chart value mismatch seen in testing (heatmap row)	Keep caption plus the original image path; add a golden question for chart values

7b. From POC to production (AWS and Azure mapping, from the flow guide)

Layer	POC	AWS	Azure
Identity	Own JWT + SQLite users	Cognito	Entra ID
API gateway and WAF	Caddy	API Gateway JWT authorizer, AWS WAF	API Management validate-jwt, Front Door WAF
Compute	One VM	ECS Fargate, Lambda	Container Apps
Async ingestion	FastAPI background task	SQS + workers	Service Bus + workers
Files and signed links	Local folder + signed URL	S3 + KMS + presigned URL	Blob + user-delegation SAS
Secrets	.env	Secrets Manager, KMS	Key Vault, managed identity
Parse and OCR	PyMuPDF + OpenAI vision	Textract	Document Intelligence
Embeddings	OpenAI text-embedding-3-small	Bedrock Titan Text Embeddings V2	Azure OpenAI text-embedding-3
Vector store	Qdrant Cloud free	OpenSearch, Aurora pgvector, or Qdrant on ECS	Azure AI Search, PostgreSQL pgvector
LLM	Groq	Bedrock	Azure OpenAI
PII	Presidio	Comprehend PII, Macie	Azure AI Language PII
Guardrails	Plain Python checks	Bedrock Guardrails	Azure AI Content Safety
Tracing and eval	Langfuse	CloudWatch, X-Ray, Langfuse	Azure Monitor, Application Insights
Infrastructure as code	Deploy script	Terraform	Terraform, Bicep

Portability: keep each service behind a small interface (IdentityVerifier, ObjectStore, VectorStore, LlmClient, Guard) with one adapter per cloud, so the pipeline never imports a cloud SDK directly. Use the same payload schema in every store. Service names and quotas change often, so confirm them in your target region (us-west-2 for the AWS capstone).

Decision to revisit: the flow guide warns that adding tenant_id or allowed_roles after ingestion means re-ingesting everything. You removed tenant_id for this POC, so a later multi-tenant or per-document access version will need a full re-ingest or a new collection. In that design the server builds a Qdrant pre-filter from the verified JWT (tenant and roles), so unauthorized chunks are never retrieved, and signed links are issued only after the same check.

8. Build order (steps 1 to 11 are built; section 0 says what is verified)

1) Settings, SQLite schema, upload and SHA-256. 2) Parse and caption pipeline with caption cache. 3) Chunking router and metadata. 4) Qdrant collection, dense and sparse upsert. 5) Query path with hybrid search. 6) Citations and guardrails. 7) Langfuse tracing. 8) Rerank switch. 9) Evaluation harness and golden questions. 10) React UI: login, upload with status, then chat with streaming and source panel, then documents and admin screens. 11) Deploy script, HTTPS, swap and nightly backup.

Next: put real keys in .env and run one document end to end; write the golden set (30 to 50 questions) and run evaluation.run; deploy to a VM and measure memory; then the Not built items.